

A MINOR PROJECT REPORT
ON
SPI MASTER CONTROLLER
SUBMITTED IN PARTIAL FULFILLMENT FOR THE AWARD OF DEGREE OF
BACHELOR OF TECHNOLOGY
IN
ELECTRONICS AND COMMUNICATION ENGINEERING



Submitted by:

KOPAL AGARWAL (9923102004)
ADYATAN AGARWAL (9923102018)
SHIVANGI SINGH (9923102035)

Under the Guidance of

DR. VIMAL KUMAR MISHRA

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA (U.P.)

April, 2026

DECLARATION

We hereby declare that this written submission represents our own ideas in our own words and where others' ideas or words have been included, have been adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Place: IIIT, Noida

Date: 29th April, 2026

Name: Kopal Agarwal

Enrollment: 9923102004

Name: Adyatan Agarwal

Enrollment: 9923102018

Name: Shivangi Singh

Enrollment: 9923102035

CERTIFICATE

This is to certify that the minor project report entitled, “**SPI Master Controller**” submitted by **Kopal Agarwal, Adyatan Agarwal, Shivangi Singh** in partial fulfillment of the requirements for the award of Bachelor of Technology Degree in **Electronics and Communication Engineering with Specialization in Computer Science** of the Jaypee Institute of Information Technology, Noida is an authentic work carried out by them under my supervision and guidance. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Signature of HOD:

Name of HOD: Dr. Arti Noor

Signature of Supervisor:

Name of the Supervisor: Dr. Vimal Kumar Mishra

ECE Department,

JIIT, Sec-128,

Noida-201304

Dated:

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to everyone who has contributed to the successful completion of this project. First and foremost, I am deeply thankful to **Dr. Vimal Kumar Mishra**, Department of Electronics and Communication Engineering, **Jaypee Institute of Information Technology**, for their invaluable guidance, constant support, and constructive feedback throughout the project. Their insights and encouragement have been instrumental in shaping this work. I also extend my gratitude to **Jaypee Institute of Information Technology** and the Department of Electronics and Communication Engineering for providing the necessary resources and a conducive environment for carrying out this project. This project has been a remarkable learning experience, and I am humbled by the opportunity to apply theoretical knowledge to a practical application.

Kopal Agarwal

Adyatan Agarwal

Shivangi Singh

ABSTRACT

The Serial Peripheral Interface (SPI) protocol is widely used in embedded systems for fast, reliable, and synchronous communication between controllers and peripheral devices. In order to combine effective data transfer with fundamental real-time processing, this project describes the design and implementation of an SPI-based communication system coupled with a data-driven warning mechanism. In order to facilitate organized and synchronized data interchange between master and slave devices, the system uses Verilog HDL to create an SPI Master Controller. Standard signals like MOSI, MISO, Serial Clock (SCLK), and Chip Select (CS) are used.

The architecture uses a modular design approach and includes essential parts including a bit counter, shift register, clock divider, and finite state machine (FSM). The FSM regulates the order of operations, including data loading, transmission, and completion, while the clock divider creates the necessary SPI clock from the system clock. The bit counter guarantees precise tracking of sent bits, while the shift register permits serial data transfer. To mimic sensor-like behavior and verify system functionality, a dummy slave module is included. The design is first validated using simulation in Vivado, where waveform analysis verifies adequate control sequencing, timing synchronization, and data transfer under various test situations.

Following successful verification, Cadence Genus is used to synthesize the RTL design and provide an optimal gate-level netlist while taking timing, area, and power restrictions into account. After that, Cadence Innovus imports the synthesized design for physical execution. To create the final physical layout in Innovus, the design goes through crucial phases like floorplanning, placement, clock tree synthesis, and routing. This entire RTL-to-GDSII pipeline guarantees that the design is both physically feasible and functionally correct, reflecting industry-standard chip design principles.

A threshold-based alarm mechanism, which compares the received data to a predetermined limit, is implemented to improve system functioning beyond simple communication. A warning signal is sent if the data surpasses the threshold, allowing for straightforward real-time monitoring and decision-making. Applications in embedded systems, sensor interfacing, and Internet of Things-based monitoring solutions might benefit from this project's successful integration of communication protocols with fundamental data processing and complete VLSI design processes.

TABLE OF CONTENTS

DECLARATION	i
CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
1. INTRODUCTION	1
1.1 Background	2
1.2 Problem Statement and Objectives	3
1.3 Social and Environmental Relevance	4
1.4 Project Plan	5
1.4.1 Work Breakdown Structure	5
1.4.2 Timeline	6
1.4.3 Roles and Responsibilities	7
1.4.4 Challenges and Risk Management	7
1.5 Scope and Limitations	8
2. LITERATURE SURVEY	9
2.1 100 MHz High Speed SPI Master	10
2.2 SPI Module using FPGA Design Flow	11
2.3 Design and Verification of SPI	12
2.4 Single Master Single Slave SPI	13
2.5 RTL Optimization in SPI Master	14
2.6 ASIC Synthesis Flow	15
2.7 ASIC Design Flow and Methodology	16
2.8 RTL to GDSII RISC-V Implementation	17
2.9 Ternary Logic Synthesis	18
2.10 Advanced RTL Optimization	19

SUMMARY TABLE	20
RESEARCH GAP	21
3. SYSTEM DESIGN AND METHODOLOGY	22
3.1 Design Considerations	23
3.2 Methodology	24
3.3 Modelling	25
3.4 Tool and Technique Selection	26
4. IMPLEMENTATION AND TESTING	27
4.1 Development Details	27
4.2 Experimental Setup	28
4.3 Test Cases / Case Study	29
4.4 Safety Measures	30
5. RESULTS AND DISCUSSION	31
5.1 Result Analysis	31
5.2 Comparison	32
5.3 Comparison with Existing Solutions	33
6. CONCLUSION AND FUTURE SCOPE	34
6.1 Summary of Achievements	34
6.2 Impact on Society, Environment and Ethics	35
6.3 Limitations	36
6.4 Future Work	37
REFERENCES	38
APPENDIX	39

LIST OF TABLES

Table No.	Caption	Page No.
1.1	Project Timeline	4
2.1	Summary of Literature Review	17
5.1	Comparison with Existing Solutions	29

LIST OF FIGURES

Figure No.	Caption	Page No.
1.1	SPI Communication between Master and Slave Devices	1
1.2	Block Diagram of Proposed SPI-Based System	3
2.1	RTL View of SPI Master	7
2.2	SPI Data Transmission	8
2.3	Verification Environment	9
2.4	Single Slave Single Master SPI	9
2.5	SPI Modes	10
2.6	Steps involved in synthesis flow using Genus Synthesis Solution tool by Cadence	11
2.7	Modern ASIC design flow	12
2.8	Top level synthesis of 32- bit RISC V CPU Core	13
2.9	The process of our device-independent ternary logic synthesis based on GT-LOGIC.	14

2.10	Proposed Hybrid-ML Guided Synthesis Methodology	14
3.1	End-to-End Workflow of SPI based Data Processor	19
3.2	RTL Schematic of SPI-Based System	21
4.1	Simulation waveform showing correct SPI communication signals	24
4.2	RTL schematic showing module connections	25
5.1	Final routed layout of the SPI-based system	26
5.2	Clock Tree Synthesis (CTS)	27
5.3	Area Report	27
5.4	Timing Report	28
5.5	Power Report	28

LIST OF ABBREVIATIONS

Abbreviation	Full Form
SPI	Serial Peripheral Interface
MOSI	Master Out Slave In
MISO	Master In Slave Out
SCLK	Serial Clock
CS	Chip Select
FSM	Finite State Machine
RTL	Register Transfer Level
HDL	Hardware Description Language
VLSI	Very Large Scale Integration
ASIC	Application Specific Integrated Circuit
FPGA	Field Programmable Gate Array
EDA	Electronic Design Automation

PPA	power, Performance, and Area
CTS	Clock Tree Synthesis
STA	Static Timing Analysis
GDSII	Graphic Data System II (Chip Layout Format)
IoT	Internet of Things

CHAPTER 1

INTRODUCTION

To operate effectively and dependably, various equipment in contemporary electronic systems must connect with one another. For instance, in order to monitor values, regulate processes, and make choices, microcontrollers, sensors, and other digital components continuously communicate data. The Serial Peripheral Interface (SPI), which enables a master device to send and receive data from one or more slave devices, is one of the most widely used communication techniques for this purpose. Compared to other communication protocols, SPI is extensively used because it is easy to build, provides high-speed transmission, and only requires a few signals. It is frequently encountered in Internet of Things (IoT) devices, embedded systems, sensor-based applications, and microcontroller-based architectures.

SPI's full-duplex capabilities, which allows data to be delivered and received simultaneously and speeds up and improves communication, is another significant benefit. Maintaining clarity and simplifying communication is made easier by using different lines for data transmission and reception. These benefits make SPI a popular choice in systems where simplicity and performance are crucial. Even though SPI is a straightforward concept, it is not always easy to create a system that guarantees accurate data transfer, appropriate timing, and signal synchronization. To prevent mistakes and guarantee dependable operation, thorough control logic implementation and appropriate coordination between various modules are necessary.

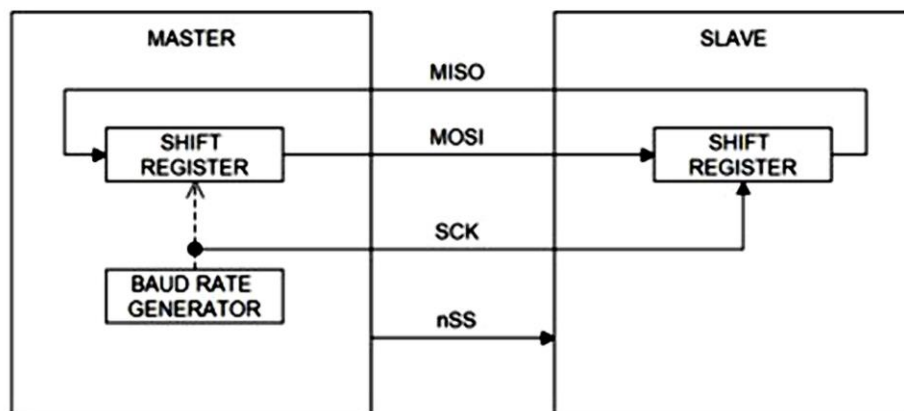


Fig 1.1 SPI Communication between Master and Slave Devices

1.1 Background

In modern electronic systems, there is an increasing need for fast, reliable, and efficient communication between components such as sensors, microcontrollers, and processors. Systems today are expected not only to transfer data but also to respond quickly to changing conditions. This makes communication protocols like SPI very important, as they provide a simple and high-speed way for devices to exchange data in real time.

SPI is popular because it uses few signals to enable direct communication between a master and one or more slave devices. It facilitates full-duplex communication, which increases efficiency by allowing data to be sent and received simultaneously. SPI is frequently utilized in embedded systems, sensor interfaces, and control applications where fast and dependable data transfer is necessary because of these benefits.

However, merely transporting data is insufficient in many real-world applications. Systems must keep an eye on circumstances and act when anomalous data is found, for as by sending out alarms in environmental, industrial, or healthcare systems. The majority of conventional SPI systems simply concentrate on communication and lack such processing capabilities. The primary driving force behind this project is the requirement for systems that integrate communication with fundamental data processing and decision-making.

1.2 Problem Statement and Objectives

Most existing SPI systems focus only on basic communication between devices. They do not include mechanisms to verify, analyze, or respond to the received data. The design of an SPI-based system involves several technical challenges that affect reliability and performance. The key problems addressed in this project are:

- Ensuring proper synchronization between clock (SCLK) and data signals (MOSI/MISO) to avoid incorrect data transfer
- Achieving accurate bit shifting at the correct clock edge during serial communication
- Coordinating multiple modules such as FSM, shift register, and bit counter for correct sequencing
- Preventing errors such as incomplete data transfer or incorrect bit counting
- Lack of data processing and alert generation in traditional SPI systems

Objectives

- To design a synchronized SPI Master Controller using Verilog HDL
- To ensure accurate 8-bit data transfer using shift register and bit counter
- To implement FSM-based control for proper sequencing of operations
- To maintain correct timing using clock edge-based data shifting
- To generate an alert signal based on threshold comparison of received data

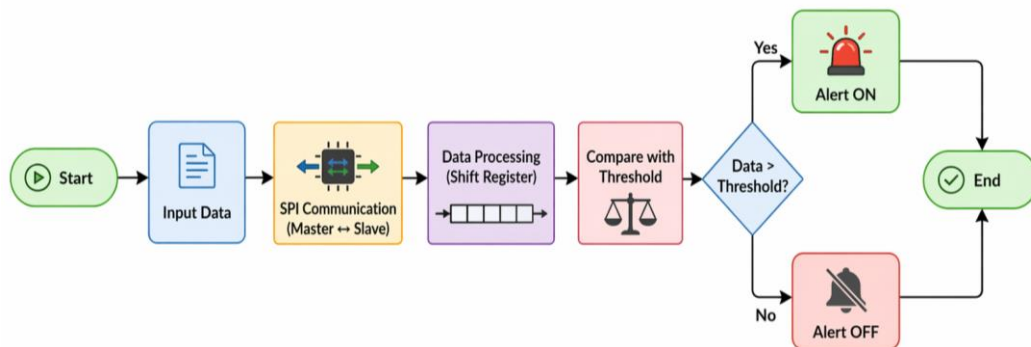


Fig 1.2 Block Diagram of Proposed SPI-Based System

1.3 Social and Environmental Relevance

The proposed system can be used in various real-world applications where monitoring and quick response are important. In healthcare systems, it can help detect abnormal conditions and generate alerts. In industrial systems, it can improve safety by identifying faults early. In environmental monitoring, it can track parameters like temperature and pollution levels and provide warnings when limits are exceeded.

From an environmental perspective, the design focuses on low power consumption, which helps in reducing energy usage. Efficient digital systems contribute to sustainable technology by minimizing power usage and improving system performance.

1.4 Project Plan

This project plan shows how we divided the work into smaller tasks and completed them one by one. It also helps in keeping track of progress and making sure everything is done on time. By planning in this way, we were able to organize our work better and avoid confusion during the project.

1.4.1 Work Breakdown Structure

- Study SPI protocol and communication basics
- Design system architecture and module structure
- Develop RTL code using Verilog
- Integrate modules into top-level design
- Perform simulation and debugging
- Perform synthesis using Cadence Genus
- Perform physical design using Cadence Innovus
- Analyze results and prepare documentation

1.4.2 Timeline

Phase	Task Description
Phase 1	Literature Review and Understanding SPI Protocol
Phase 2	System Design and Architecture Planning
Phase 3	RTL Design of Modules
Phase 4	Integration of Modules and Testing
Phase 5	Simulation and Verification using Vivado
Phase 6	Debugging and Validation
Phase 7	Synthesis using Cadence Genus
Phase 8	Physical Design using Cadence Innovus
Phase 9	Result Analysis and Documentation

Table 1.1 Project Timeline

1.4.3 Roles and Responsibilities

- Designing RTL modules and writing Verilog code
- Performing simulation and debugging
- Running synthesis and analyzing reports
- Performing physical design steps
- Preparing documentation and report

1.4.4 Challenges and Risk Management

The project involved challenges related to synchronization, control logic, and module integration. Maintaining correct timing between clock and data signals was important for proper SPI communication. Coordinating modules like the FSM, shift register, and bit counter also required careful design and testing to avoid errors in data transfer.

During synthesis and physical design, issues such as timing errors and tool-related problems were encountered. These were managed by testing each module step by step, using simulation to verify functionality, and analyzing reports to fix errors. This approach helped in reducing risks and ensuring correct system operation.

1.5 Scope and Limitations

The design and implementation of an SPI Master Controller that can reliably communicate with a slave device utilizing common SPI signals is part of the project's scope. In order to facilitate basic real-time data monitoring, the system also includes a threshold-based alarm mechanism. A full hardware design flow is demonstrated by extending the design to synthesis and actual implementation once it has been validated through simulation.

The system does, however, have some drawbacks. Practical validation is limited since it employs a dummy slave module rather than actual hardware sensors. The alarm system lacks sophisticated data analysis methods and is predicated on a set threshold. Furthermore, the design does not enable sophisticated SPI modes or numerous slave devices; instead, it concentrates on a simple SPI configuration. Notwithstanding these drawbacks, the project offers a solid basis for comprehending system-level architecture and SPI communication.

CHAPTER 2

LITERATURE SURVEY

The development of this SPI-based data processing and alert system is based on a study of existing research in SPI communication and digital design. This literature survey helps in understanding how earlier systems were designed and what limitations they had. Many works focus only on communication or individual design stages and do not cover the complete flow from RTL to physical implementation. The review includes studies on SPI design, verification, and ASIC design flow using tools like Vivado, Cadence Genus, and Innovus.

The objective of this survey is to understand SPI communication and the overall hardware design process. It also helps in identifying gaps such as lack of data processing and alert mechanisms in existing systems. Based on these observations, this project aims to combine communication with simple processing and demonstrate a complete RTL to GDSII design flow.

2.1 100 MHz High Speed SPI Master: Design, Implementation and Study on Limitations of using SPI at High Speed ^[1]

In this study, a high-speed SPI master core based on FPGA is shown, with a focus on both practical high-frequency design restrictions and standard protocol principles. It starts by highlighting SPI's dominance in embedded systems and justifies it as a straightforward, full-duplex, synchronous serial bus that permits higher throughput than UART and I2C due to the lack of start/stop and acknowledgment overhead. According to the abstract, the paper's main objectives are to design and develop a 100 MHz high-speed SPI master core on FPGA using a state-machine-based RTL method, as well as to investigate the difficulties and constraints associated with employing SPI at such high rates in digital systems. In order to provide a specific condition on the maximum safe SPI frequency for a particular board and device configuration, the authors derive a timing inequality that links the SCLK period to the propagation delay of the clock line, the slave's internal clock-to-output delay, the propagation delay of the MISO line, and the setup time at the master. The work serves as both an implementation document and a design guideline for high-speed SPI lines in actual hardware. They then go over practical solutions to this constraint, like shorter interconnects, faster slaves, and the use of buffers to reduce effective RC delay.

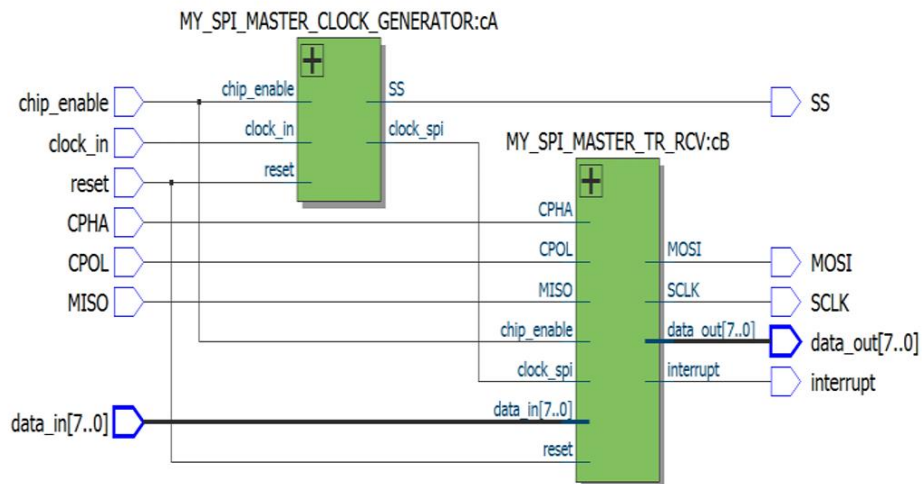


Fig 2.1 RTL View of SPI Master

2.2 Design and Implementation of SPI Module in Verilog HDL using FPGA Design Flow [2]

In this work, a conventional FPGA design flow is used to develop and implement a Serial Peripheral Interface (SPI) communication module in Verilog HDL. The authors stress in the introduction that SPI is a short-range serial bus created by Motorola that can operate at extremely high rates, be configured as master or slave, scale up to about 30 slave devices, and have transmit/receive register widths that can be adjusted beyond the standard 8 or 16 bits. This paper's unique selling point is that, rather than merely providing a conceptual description of SPI, it explicitly walks through and applies the whole FPGA design flow for an SPI module, making it a practical implementation and instructional resource. Every stage is described in detail, including the design definition, behavioral description, Verilog RTL description, functional verification via simulation, logic synthesis into a gate-level netlist, placement and routing, programming, and testing on an actual Virtex-5 FPGA. For students or designers who are new to SPI or FPGA-based IP core design, this step-by-step flow, along with real master and slave RTL schematics and hardware verification, provides readers with a useful template for taking a communication protocol from specification to functional FPGA prototype.

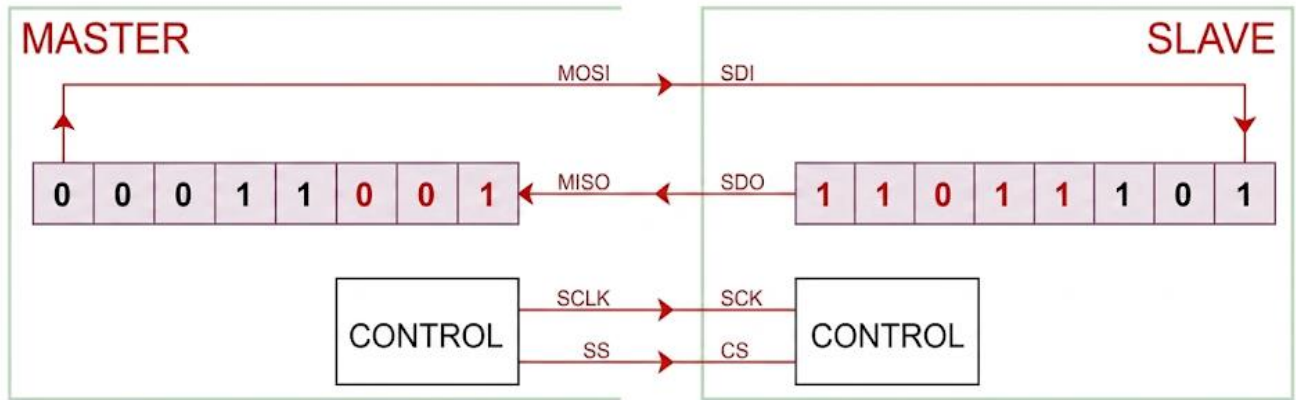


Fig 2.2 SPI Data Transmission

2.3 Design and Verification of Serial Peripheral Interface [3]

The design and verification of a Serial Peripheral Interface (SPI) IP core intended for contemporary ASIC/SoC applications—specifically, on-board communication between a processor and several peripherals—are the main topics of this article. In order to determine the fundamental characteristics required in a reusable, universal SPI core, it begins with the standard Motorola SPI specification and examines commercial SPI devices. The introduction promotes SPI as a straightforward, high-speed, low-pin-count protocol for short-distance communication and highlights the lack of a comprehensive treatment that begins with Motorola's SPI bus specification and continues all the way down to concrete ASIC/FPGA implementation with full architectural and design details, despite a wealth of literature dating back to the 1980s. In order to solve this, the authors construct a universal SPI IP core using a design-reuse technique after conducting a market assessment of current commercial SPI devices to extract typical requirements. The authors demonstrate 100% code coverage and complete functional coverage of the SPI design, and the verification environment is specifically designed for reuse as a verification IP employing object-oriented features and restricted randomization to hit corner cases. Additionally, they show that the IP is not only academic but also deployment-ready by integrating the developed SPI core into a genuine SoC (with a PPC440 CPU and other peripherals) for avionics applications.

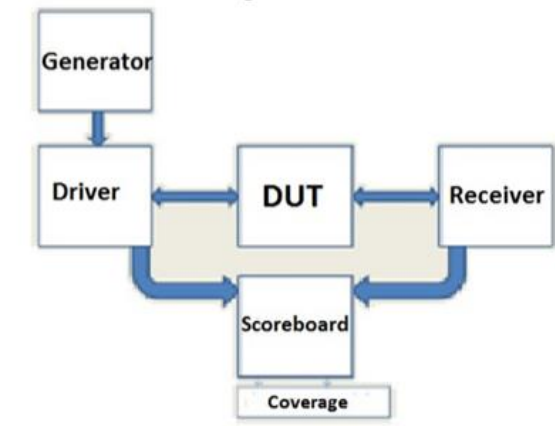


Fig 2.3 Verification Environment

2.4 Design and Implementation of Single Master Single Slave Serial Peripheral Interface (SPI) on FPGA [4]

According to the abstract, I2C and SPI are the most popular low-end communication protocols for on-board peripheral communication. This work presents a "high-quality" SPI IP with one-master/one-slave configuration and 8-bit data transfer, which includes all the features needed by contemporary ASIC/SoC applications. With the RTL design written in Verilog for synthesis and the verification environment written in SystemVerilog, the designed SPI core is meant to connect a CPU to multiple peripherals in a SoC, particularly for low-/medium-speed data stream transmission. Simulations verify full-duplex operation and proper mode behavior, the design is synthesized and mapped onto a Xilinx Virtex-5 FPGA, and coverage reports show 100% code coverage and full functional coverage, indicating that the SPI IP is reliable enough to be integrated into actual SoC applications.

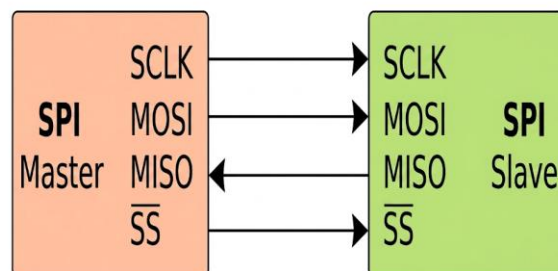


Fig 2.4 Single Slave Single Master SPI

2.5 Design and Optimization in SPI Master at the RTL Level [5]

A straightforward 4-wire interface (SCK, MOSI, MISO, SS) and adjustable clock polarity and phase via CPOL and CPHA bits are the foundations of SPI, a synchronous, full-duplex, short-distance communication protocol. In order to regulate data transit between the controller and the host circuitry, the SPI master offers well-defined TX and RX interfaces at the top level. These interfaces use handshaking signals like data-valid and ready. In systems where the SPI clock domain differs from the system clock domain, this is crucial because it separates the SPI timing from the system bus, enabling the master to buffer data and start transfers only when both sides are ready. All four common SPI modes can be used without changing the hardware data path thanks to the control interface's configurable registers for CPOL and CPHA. Shift-register methods for both MOSI and MISO are the foundation of the data path. While receiving serial data on MISO is concurrently shifted into a receive shift register and presented to the RX interface once a whole word has been collected, outgoing parallel data is loaded into a transmit shift register and shifted out bit-by-bit on MOSI in time with the SPI clock.

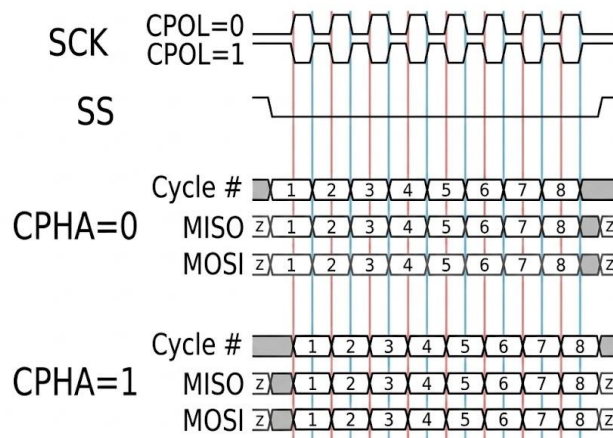


Fig 2.5 SPI Modes

2.6 A Review on ASIC Synthesis Flow Employing Two Industry Standard Tools [6]

The Application Specific Integrated Circuit (ASIC) synthesis flow is compared in this study, with an emphasis on the usage of industry-standard Electronic Design Automation (EDA) tools such as Cadence Genus Synthesis Solution and Synopsys Design Compiler. Three basic steps make up the synthesis

process: translation, mapping, and optimization. These steps balance power, performance (timing), and area (PPA) while converting Register Transfer Logic (RTL) instructions into an efficient gate-level netlist. The paper describes a process that starts with gathering input files (RTL code, technology libraries, and restrictions), elaborates into cells that are independent of technology, and ends with gate-level optimization.

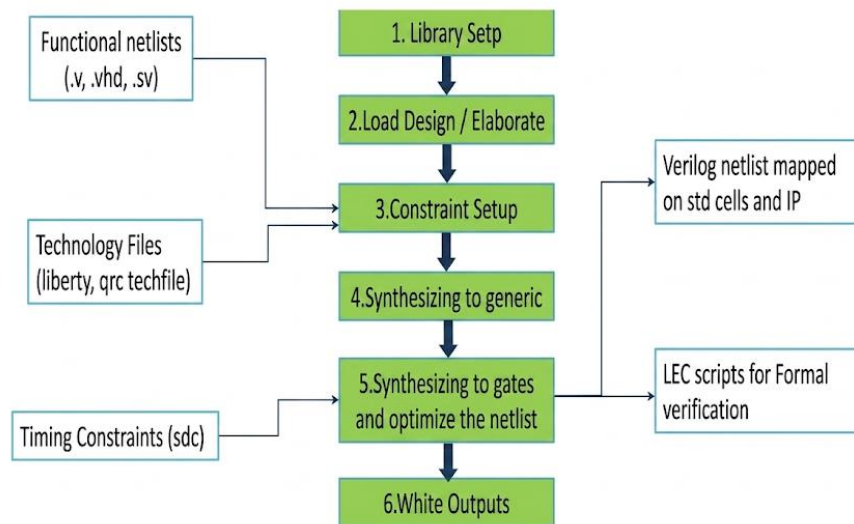


Fig 2.6 Steps involved in synthesis flow using Genus Synthesis Solution tool by Cadence

2.7 ASIC Design Flow And Methodology [7]

Designing an ASIC (a custom computer chip built for a specific task) is a complex journey that transforms a high-level idea into a physical piece of silicon through a systematic "flow". Because these chips contain millions of transistors, engineers cannot design them manually; instead, they use a structured process divided into specialized stages starting with the Specification phase, where the chip's purpose, speed, and power requirements are defined. This is followed by writing a digital blueprint in a specialized computer language called RTL, which undergoes rigorous Verification and Linting to catch logic errors and common mistakes before any physical work begins. Once the digital logic is solid, it enters Synthesis, where abstract code is converted into a detailed list of actual electronic components like gates and transistors, and Design for Testability (DFT) features are added to allow the factory to check for manufacturing defects later.

The design then moves into the Physical Design stage, which is like creating a microscopic floor plan to decide the exact location of millions of parts and how to connect them with miles of tiny wiring. Before

being sent to the factory, the chip must pass final "health checks," including Static Timing Analysis (STA) to ensure electrical signals move fast enough and Physical Verification to confirm the layout matches the original blueprint and follows all manufacturing rules.

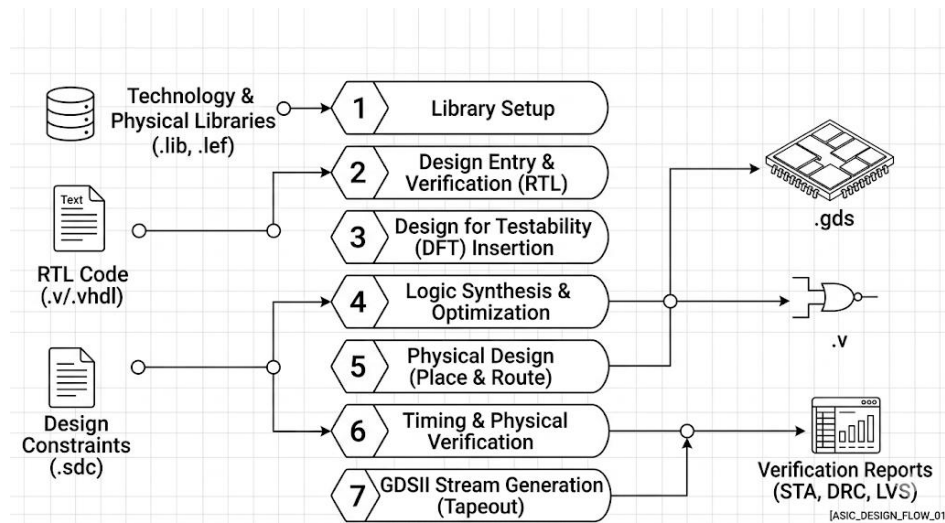


Fig 2.7 Modern ASIC design flow

2.8 Design and Physical Implementation of a 32-bit RISC-V Processor using RTL to GDSII Flow [8]

The complete physical implementation of a 32-bit RISC-V processor is shown in this study, which manages the challenging transition from high-level Verilog RTL to the final GDSII layout. The researchers constructed a typical cell-based flow on a 45nm TSMC technology library using Cadence Innovus and Genus tools. To guarantee high throughput, the core design uses a 5-stage pipeline (Fetch, Decode, Execute, Memory, Write-back) in accordance with the RV32I instruction set. Floorplanning with a goal utilization of 70%, Clock Tree Synthesis (CTS) to reduce skew, and global routing to address congestion were important physical design milestones. The final results showed a reliable technique for low-power embedded systems, achieving a target frequency of 200 MHz while maximizing the Power, Performance, and Area (PPA) parameters.

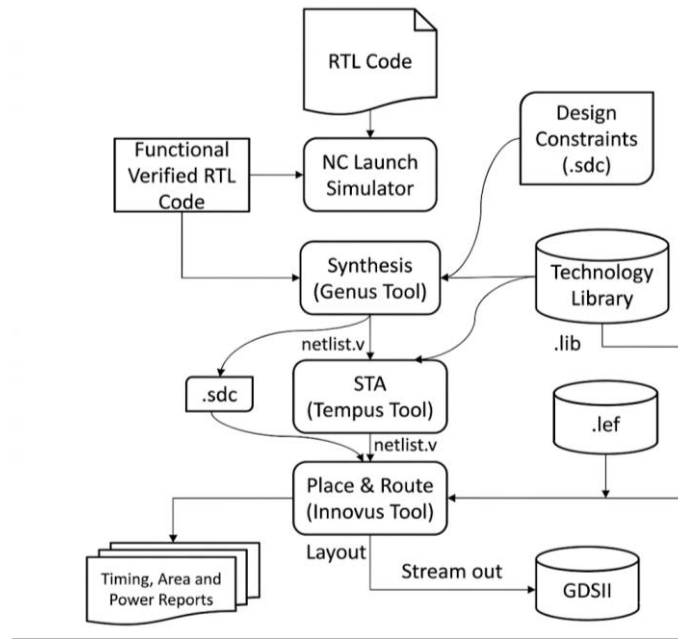


Fig 2.8 Top level synthesis of 32- bit RISC V CPU Core

2.9 An RTL-Based General Synthesis Methodology for Device-Independent Ternary Logic Circuits [9]

The construction of a 32-bit RISC-V processor, an open-source "brain" for electronic devices, is the subject of this research study. The researchers employed a specialized design flow (RTL-to-GDSII), which functions as a highly sophisticated architectural process, to make this processor function well while consuming very little battery life. They took the original digital concept and meticulously mapped out exactly where millions of tiny parts should sit on a microscopic piece of silicon. They were able to reduce power usage by about 16% without slowing down the chip's speed by dividing the chip's operations into five easy steps and employing intelligent tools to control the timing of electrical impulses.

Finally, the study went beyond today's standard technology by comparing this design to "future" materials like carbon nanotubes and memristors, which are experimental components that could one day replace traditional silicon to make our smartphones and computers even faster and more energy-efficient.

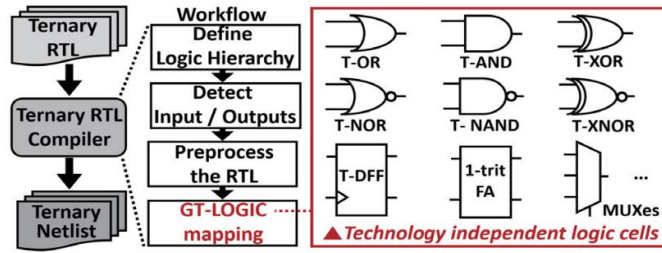


Fig 2.9 The process of our device-independent ternary logic synthesis based on GT-LOGIC.

2.10 Advanced RTL Optimization and Logic Synthesis Strategies for High-Performance VLSI Systems [10]

The goal of the research article "Advanced RTL Optimization and Logic Synthesis Strategies for High-Performance VLSI Systems" is to reduce the size, speed, and energy consumption of the computer chip, which is the "brain" of contemporary electronics. This work investigates the use of machine learning and clever mathematical techniques to more efficiently translate digital code (RTL), which engineers create before a chip is physically built, into a physical circuit. The researchers discovered a way to drastically cut the physical area and power consumption of chips by breaking down huge, complex designs into smaller "mini-projects" and using AI to anticipate which design choices will work best.

Essentially, it's like having a super-intelligent assistant who knows exactly how to pack a massive amount of gear into a tiny suitcase without making it heavy or disorganized. This work is a big deal for the future of gadgets like smart watches and high-end smartphones, as it ensures they can handle more powerful apps while keeping the battery life from draining too quickly.

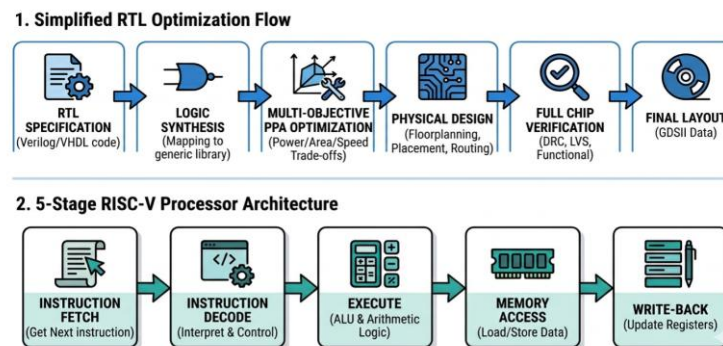


Fig 2.10 Proposed Hybrid-ML Guided Synthesis Methodology

SUMMARY TABLE

S.No.	Research Paper	Methodology/Approach	Efficiency & Key Improvements	Main Limitations
1	100 MHz SPI Master	FSM-based SPI master in HDL, implemented and timed on Cyclone III FPGA	Reaches 100 MHz SCLK (max 119 MHz) and derives a timing bound for high-speed SPI	No detailed area/power optimisation, only single-slave, fixed 8-bit, with basic verification
2	SPI Module on FPGA	Verilog master–slave SPI designed and taken through full Xilinx FPGA design flow.	Demonstrates correct full-duplex SPI on Virtex-5, good as a teaching/reference design	No high-speed target, optimization, or advanced/handshaked system interface
3	SPI Design & Verification	SPI master–slave verified with SystemVerilog coverage-driven testbench	Achieves functional coverage with correct full-duplex operation	Little focus on RTL optimization or scalability beyond single-master, single-slave, 8-bit

4	Single-Master Single-Slave SPI on FPGA	FPGA implementation of one-master/one-slave SPI link with protocol-correct signalling	Validates stable, full-duplex communication across SPI modes in hardware	Almost no discussion of resource optimisation, advanced features, or complex system integration.
5	RTL-Optimized SPI Master	Compares alternative SPI-master RTL architectures via synthesis-based evaluation	Improves timing versus baseline SPI masters through structural optimisation	Optimisation is core-centric, with limited treatment of multi-slave support or rich SoC-level interfaces
6	RTL-to-GDSII for RISC-V Processor	Uses a 5-stage pipeline and Synopsys Fusion Compiler on 32nm tech.	15.9% reduction in total power while maintaining 200MHz speed.	Specifically focused on one processor type; results may vary on larger chips.
7	Ternary Logic Synthesis (KNU)	A new "three-state" language based on Verilog to automate chip design.	63% fewer internal cells, drastically shrinking the chip's physical size.	Requires a specialized logic library that isn't standard in the industry yet.

8	Advanced RTL & ML Optimization	Combines Machine Learning with breaking big designs into smaller chunks.	Significantly better Quality of Results (QoR) by predicting the best design paths.	The AI models require a lot of data and computing power to train initially.
9	ASIC Synthesis Flow Review	Comparison of industry tools (Synopsys vs. Cadence) for chip design.	Identifies which specific settings optimize power and area the most.	Mostly a review of existing work rather than creating a new physical chip.
10	ASIC Flow Overview	A step-by-step guide connecting all engineering roles in chip creation.	Higher educational value; helps prevent errors by explaining the "big picture."	Does not provide new data or specific experimental improvements.

Table 2.1 Summary of Project Review

RESEARCH GAP

Based on the analysis of the research papers, the "research gaps" can be broken down into these key points:

- Current efforts rarely optimize for all three—Power, Performance, and Area (PPA)—at once; instead, they frequently concentrate on a single objective, such as lowering power or area. Furthermore, they frequently lack the thorough, industrial-style verification required to demonstrate that the design functions flawlessly in the actual world.
- Most designs use very basic connections that make them difficult to "drop" into a larger, complex system. They often miss professional features like data buffers (FIFOs), synchronized handshakes (valid/ready signals), or the ability to move data safely between different clock speeds.
- High-level features that ensure robust communication—such as adaptive baud-rate control, explicit error handling (timeouts, parity, or CRC), and intentional low-power techniques like clock gating—remain largely unexplored in these specific implementations.
- While some papers mention Verilog-based testing, many rely on basic waveform simulations. There is a gap in providing a truly industrial-style verification environment that tests for corner cases, high-traffic stability, and long-term reliability within a larger integrated system.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

Reliable serial transmission, data processing, and real-time alarm creation are all made possible by the SPI-Based Data Processing and alarm System, a modular and effective digital hardware system. A clock divider, finite state machine (FSM), shift register, and bit counter are among the interconnected modules used in the system's construction. These modules cooperate to provide correct synchronization and regulated data transport. A threshold comparison mechanism that verifies the incoming data and produces an alert signal when the value above a certain limit is another feature of the system in addition to communication. The overall workflow of the system is shown in Fig. 3.1, which represents the complete sequence of operations starting from data input, followed by SPI communication, internal processing, and finally comparison with a threshold to produce an alert output.

The overall design follows a structured and modular approach, where each block performs a specific function and contributes to the complete operation of the system. This makes the design easier to understand, debug, and extend in the future. The design is first developed at the Register Transfer Level (RTL) using Verilog, then verified through simulation, followed by synthesis and physical implementation, demonstrating a complete digital design flow.

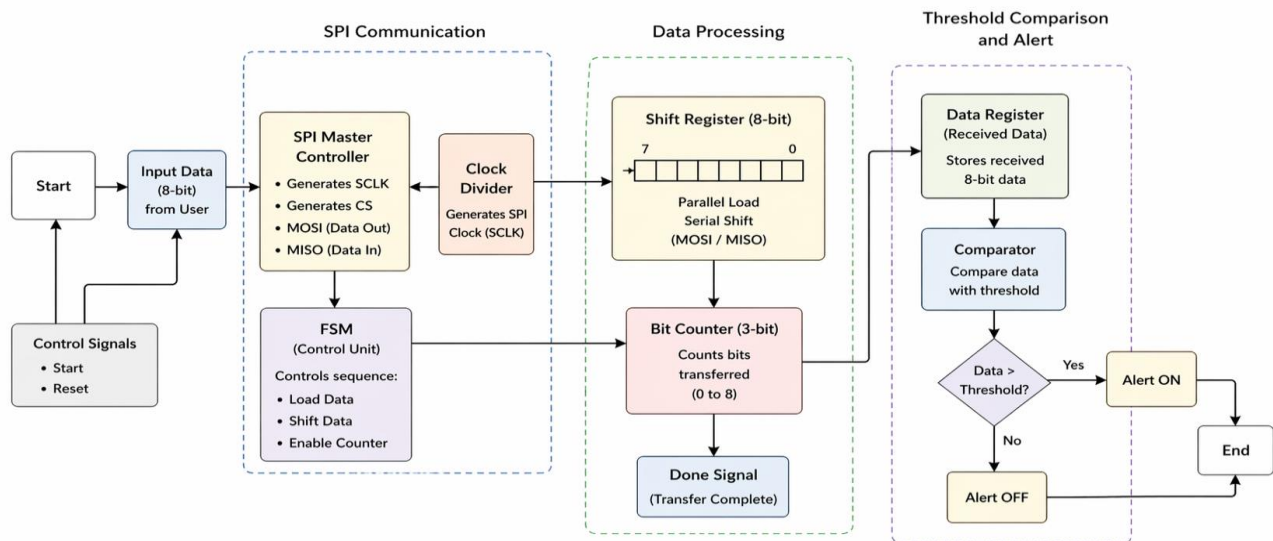


Fig. 3.1 End-to-End Workflow of SPI based Data Processor

3.1 Design Considerations

Several crucial elements were taken into account during the system's construction to guarantee that it is useful, effective, and appropriate for real-world applications. Simple and modular digital components are used in the system's construction, which lowers implementation costs and design complexity. Additionally, a modular design makes it simpler to test individual parts and make necessary system improvements.

From an environmental standpoint, the design emphasizes minimal power consumption, which promotes sustainable digital design techniques and aids in lowering energy use. The system can be used in fields where prompt response to anomalous data is crucial, like environmental sensing, industrial automation, and healthcare monitoring. By delivering timely warnings in these applications, the alert mechanism contributes to increased safety.

The design also follows ethical considerations by not involving any personal or sensitive data. It is a general-purpose digital system that can be used safely in different applications. Additionally, the use of simulation and virtual tools reduces the need for physical hardware during development, which helps in minimizing electronic waste and supports sustainability goals.

3.2 Methodology

The system is designed in a simple step-by-step manner so that each part works together smoothly. First, the input data is given to the SPI master along with control signals like start and reset. The reset signal is used to clear all the internal values and bring the system to an initial state, while the start signal begins the communication process. This ensures that the system always starts in a proper and known condition.

After the system starts, the SPI communication begins. The master device sends data to the slave one bit at a time using the SPI signals. At the same time, it also receives data from the slave, which shows full-duplex communication. The transfer of data happens in a synchronized way, meaning the data is sent and received according to the clock signal so that there are no errors.

While the data transfer is happening, the system keeps track of how many bits have been sent. Since the system works with 8-bit data, it ensures that exactly 8 bits are transmitted in one complete cycle. Once all the bits are sent and received, the system stops the communication and stores the received data for further processing.

After receiving the data, the system checks the value using a simple comparison step. The received data is compared with a fixed threshold value. If the data is greater than this value, the system generates an alert signal. If the data is within the limit, no alert is generated and the system continues its normal operation. This adds a simple decision-making feature to the system.

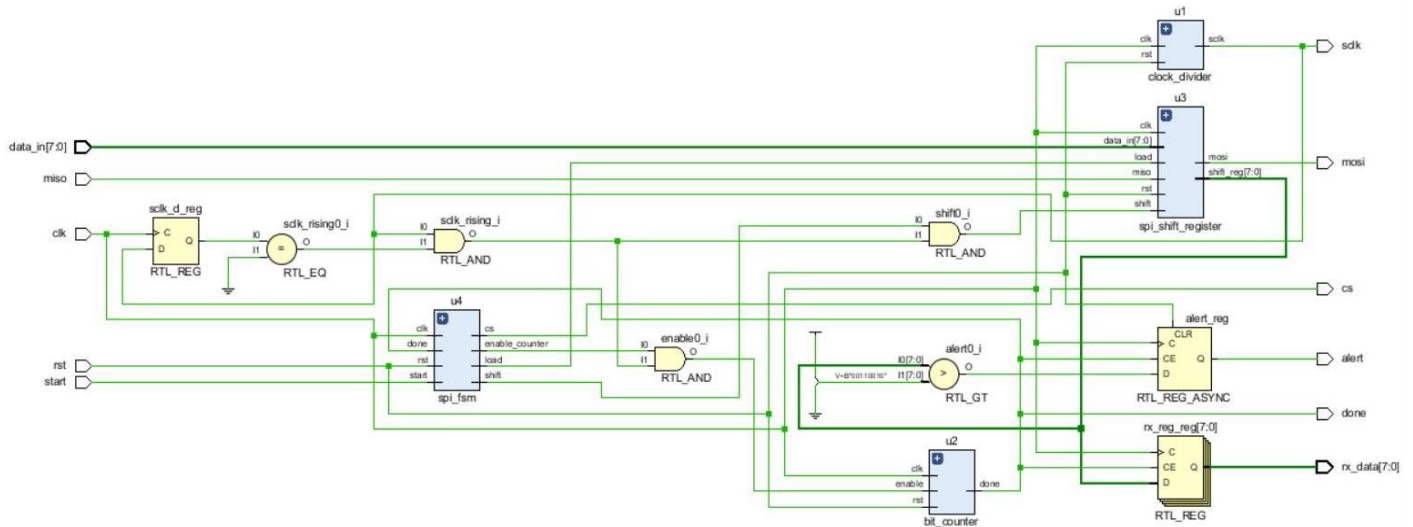


Fig 3.2 RTL Schematic of SPI-Based System

3.3 Modelling

The system is modeled at the Register Transfer Level (RTL) using Verilog, where each part of the SPI-based system is designed as a separate module. The main modules include the clock divider, FSM, shift register, and bit counter. These modules are connected together in a top module to represent the complete system. This modular approach helps in understanding how each part works and ensures that the overall system operates correctly.

The behavior of the system is first verified using simulation in Vivado. Different input values and control signals are applied to check whether the system is functioning properly. The waveforms are analyzed to confirm correct data transfer, proper synchronization of clock and signals, and correct operation of control logic. The alert mechanism is also tested by comparing the received data with a threshold value to ensure that the alert signal is generated only when required.

After functional verification, the model is taken further for implementation using industry tools. The RTL design is synthesized using Cadence Genus, where it is converted into a gate-level netlist and optimized for timing, area, and power. This synthesized design is then used in Cadence Innovus for physical implementation. In this stage, the system is mapped onto a chip layout through steps such as floorplanning, placement of cells, clock tree synthesis, and routing of connections. This modeling approach not only verifies logical correctness but also ensures that the design can be physically implemented on real hardware.

3.4 Tool and Technique Selection

The tools and techniques used in this project are selected based on their effectiveness and relevance in digital hardware design. Industry-standard software is used to ensure that the design follows a complete and practical implementation flow.

Verilog HDL is used for RTL design because it provides a clear and structured way to describe hardware behavior. It allows the system to be modeled in terms of data flow and control logic, and it supports both simulation and synthesis, making it suitable for digital system design.

Vivado is used for simulation and verification of the design. It provides detailed waveform analysis, which helps in understanding signal behavior and identifying errors in the design. Through simulation, the correctness of data transfer, synchronization, and control logic is verified before moving to the next stages.

Cadence Genus is used for synthesis, where the RTL design is converted into a gate-level netlist. During this process, the design is optimized for timing, area, and power. This step ensures that the design is efficient and ready for physical implementation.

Cadence Innovus is used for physical implementation of the design. It performs important steps such as floorplanning, placement, clock tree synthesis, and routing. These steps define how the design is physically laid out on a chip and ensure that it meets all required constraints.

CHAPTER 4

IMPLEMENTATION AND TESTING

4.1 Development Details

- Designed individual modules such as FSM, shift register, clock divider, and bit counter using Verilog
- Tested each module separately to ensure correct functionality before integration
- Integrated all modules to form the complete SPI-based system
- Simulated the design in Vivado to verify data transfer, control signals, and synchronization
- Analyzed waveforms to check correct operation and debug errors
- Performed synthesis using Cadence Genus to convert RTL into gate-level netlist
- Checked synthesis reports for timing, area, and power
- Imported synthesized design into Cadence Innovus for physical implementation
- Performed floorplanning to define chip layout
- Completed placement, clock tree synthesis, and routing
- Generated final layout representing physical implementation of the system

4.2 Experimental Setup

In this project, we designed and implemented an SPI-Based Data Processing and Alert System using a complete VLSI design flow. First, the design was written in Verilog at the RTL level, where different modules like the clock divider, FSM, shift register, and bit counter were created and connected together to form the full system. After writing the code, we tested the design in Vivado using simulation. In this step, we checked whether signals like clock, start, reset, MOSI, MISO, and chip select were working properly and whether data was being transferred correctly. This helped us confirm that the system logic was correct before moving to the next steps.

After successful simulation, the design was synthesized using Cadence Genus, where the RTL code was converted into a gate-level design using standard cells. We also got reports related to area, timing, and power, which helped us understand how efficient the design is. Then the design was taken to Cadence Innovus for physical implementation. In Innovus, we performed floorplanning to define the chip area, followed by power planning to ensure proper power distribution. After that, placement was done to arrange the cells, and clock tree synthesis was performed to distribute the clock signal evenly. Finally, routing was done to connect all components using metal layers. At every stage, the design was checked carefully to make sure it was working correctly and meeting all conditions.

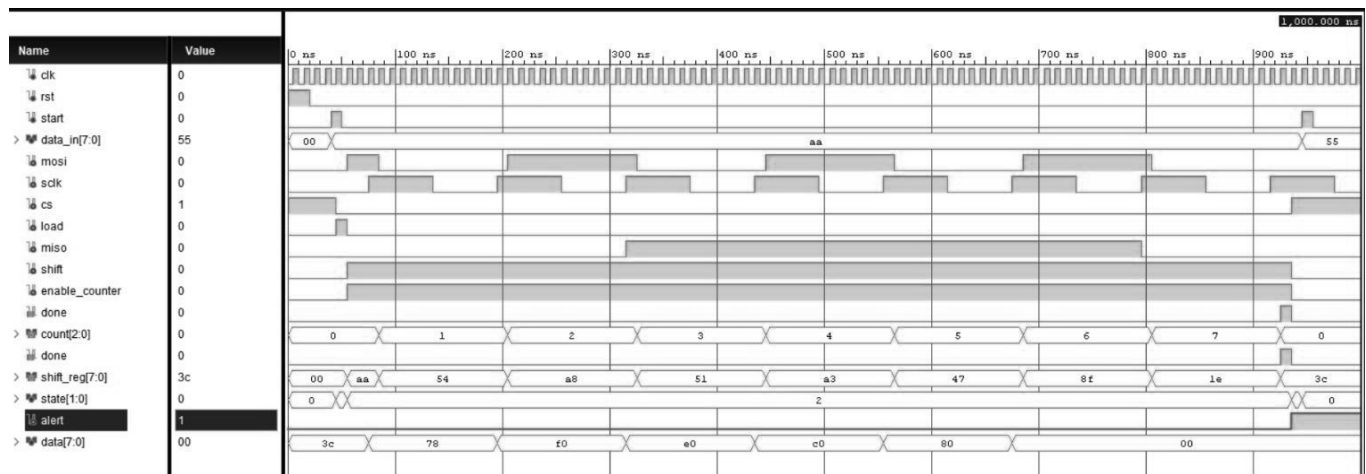


Fig 4.1 Simulation waveform showing correct SPI communication signals

4.3 Test Cases / Case Study

To test the system, we provided an 8-bit input data along with control signals such as start and reset. When the start signal is given, the system begins operation, and the FSM controls the sequence of steps. The input data is loaded into the shift register, and the clock divider generates the SPI clock, which controls the timing of data transfer. The system sends data bit by bit through the MOSI line and receives data through the MISO line at the same time, which shows that full-duplex communication is working correctly.

The bit counter keeps track of the number of bits transferred and ensures that exactly eight bits are sent during each operation. Once all 8 bits are transmitted, the system generates a done signal to indicate that the communication process is complete. After this, the received data is compared with a predefined threshold value. If the received data is greater than the threshold, the alert signal is turned ON; otherwise, it remains OFF. The simulation results clearly show that all modules are working together properly and

in synchronization, and the correct alert signal is generated at the right time.

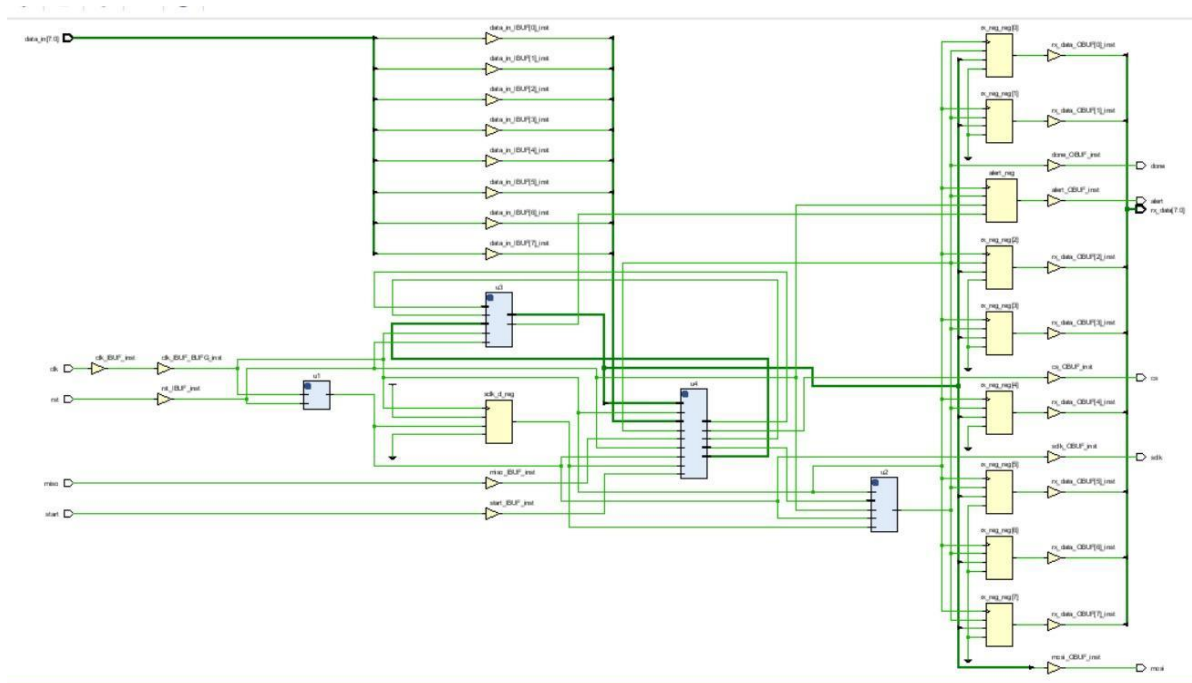


Fig 4.2 RTL schematic showing module connections

4.4 Safety Measures

During the development of this project, proper safety and security measures were considered to ensure reliable and responsible design practices. Since the project is implemented using software tools and simulation environments, there are no direct physical safety risks. However, care was taken to ensure correct handling of design files and system resources.

From a technical perspective, proper design practices were followed to avoid errors such as incorrect signal connections, timing issues, and unstable outputs. Regular testing and verification were performed to ensure that the system operates correctly under different conditions. This helps in preventing unexpected failures when the design is implemented.

In terms of cybersecurity, the project files and design data were handled securely within the development environment. No external or sensitive data was used in the project, which reduces the risk of data misuse. Additionally, the use of licensed and standard software tools ensures that the design process is secure and reliable.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Result Analysis

After completing all stages of the design, the results from synthesis and physical implementation show that the system is efficient and well designed. From the synthesis results, it is observed that the design uses around 73 standard cells, with the shift register occupying the largest area because it stores and shifts data. Other modules like the FSM and control logic use less area, which shows that the design is balanced and modular.

In timing analysis, the slack is positive (around 4 ns), which means the design meets all timing requirements and there are no violations. The routing results show no congestion or overflow, which means all connections are completed properly. Power analysis shows that the total power consumption is low (around 0.07 mW), with most of the power used by sequential elements and very little leakage power. Overall, the design performs well in terms of area, timing, and power, which shows that it is optimized and suitable for practical use.

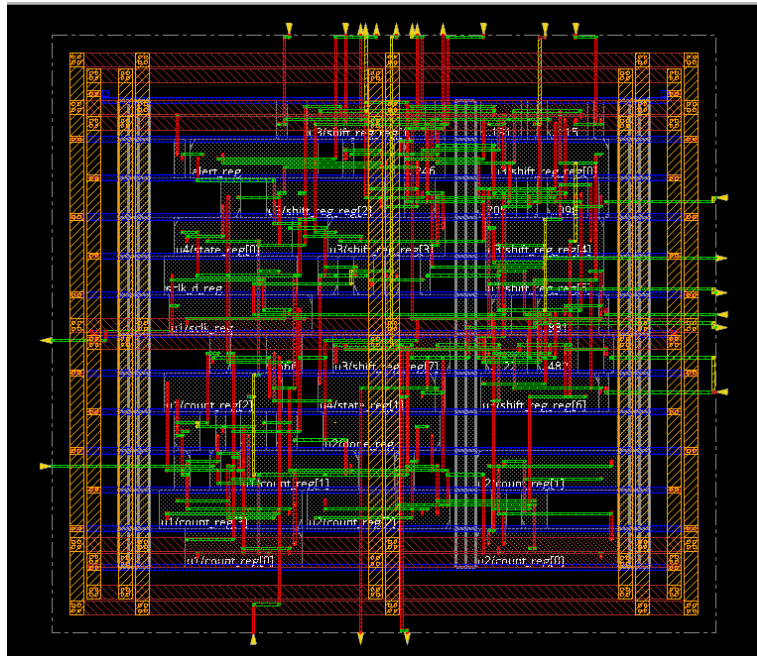


Fig 5.1 Final routed layout of the SPI-based system

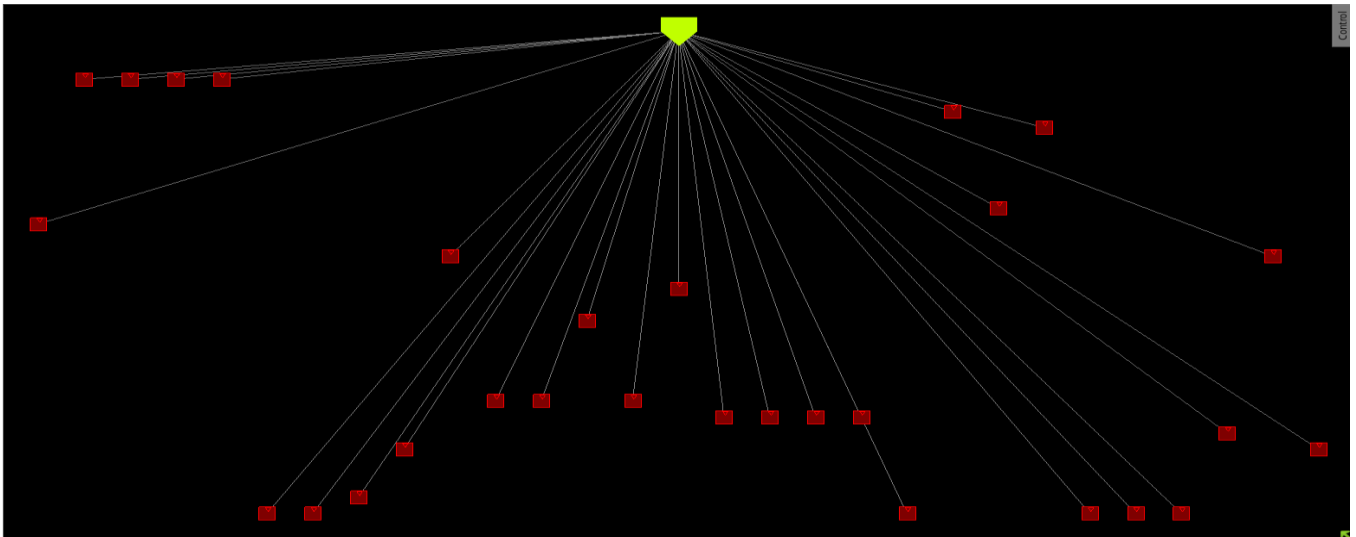


Fig 5.2 Clock Tree Synthesis (CTS)

5.2 Comparison

The performance of the system can be evaluated using parameters such as area, timing, power consumption, and routing quality. From the synthesis results, the design uses a moderate area, where the shift register occupies the largest portion due to its role in storing and shifting data.

In terms of timing, the design shows a positive slack of around 4 ns, which means there are no timing violations and all signals are working correctly. The routing results also indicate no congestion or overflow, which means the connections are properly completed. The power consumption is low (around 0.07 mW), with most of the power used by sequential elements and very little leakage power.

When compared to a basic SPI system, this design provides additional functionality by not only transferring data but also processing it and generating an alert signal. Even with this extra feature, the system remains efficient in terms of area, timing, and power, showing that the design is optimized and suitable for practical applications.

```
innovus 6> report_area
```

Hinst Name	Module Name	Inst Count	Total Area
spi_master_top		73	2349.905
u1	clock_divider	13	503.935
u2	bit_counter	7	395.842
u3	spi_shift_register	28	910.031
u4	spi_fsm	8	232.848

Fig 5.3 Area Report

Pin	Type	Fanout	Load (fF)	Slew (ps)	Delay (ps)	Arrival (ps)	
(clock clk)	launch						0 R
u4							
state_reg[0]/CK				800	+0	0	R
state_reg[0]/Q	DFFRX4	2	4.0	22	+184	184	R
g2__9945/AN					+0	184	
g2__9945/Y	NOR2BX4	2	13.3	54	+59	243	R
g170__9315/B					+0	243	
g170__9315/Y	NOR2X1	2	0.6	15	+19	262	F
u4/cs							
cs	<<< interconnect			15	+0	262	F
	out port				+0	262	F
(spi_master_top.sdc_line_19)	ext delay				+1000	1262	F
(clock clk)	capture					6000	R
	uncertainty				-10	5990	R
Cost Group : 'clk' (path_group 'clk')							
Timing slack : 4728ps							
Start-point : u4/state_reg[0]/CK							
End-point : cs							

Fig 5.4 Timing Report

Total Power:		0.06075495				
.....						
Group		Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
.....						
Sequential		0.05162	0.002805	9.326e-06	0.05443	89.59
Macro		0	0	0	0	0
IO		0	0	0	0	0
Combinational		0.003321	0.003	4.653e-06	0.006326	10.41
Clock (Combinational)		0	0	0	0	0
Clock (Sequential)		0	0	0	0	0
.....						
Total		0.05494	0.005805	1.398e-05	0.06075	100
.....						
Rail	Voltage	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
.....						
Default	0.9	0.01697	0	5.387e-06	0.01698	27.95
VDD	0.9	0.03796	0.005195	8.592e-06	0.04316	71.05
.....						
* Power Distribution Summary:						
* Highest Average Power:		alert_reg (SDFFRHQX1):				0.002131
* Highest Leakage Power:		u1/g161__4319 (NAND4X6):				7.586e-07
* Total Cap:		4.44265e-13 F				
* Total instances in design:		73				
* Total instances in design with no power:		0				
* Total instances in design with no activity:		0				
* Total Fillers and Decap:		0				
.....						

Fig 5.5 Power Report

5.3 Comparison with Existing Solutions

Parameter	Traditional SPI System	Proposed SPI System
Primary Function	Only data communication	Communication with data processing and alert generation
Control Logic	Basic or simple control	FSM-based structured control
Data Processing	Not included	Threshold-based comparison included
Alert Mechanism	Not available	Alert generated for abnormal data
Design Flow	Limited to RTL/FPGA	Complete RTL to GDSII flow
Physical Implementation	Rarely implemented	Implemented using Cadence Innovus
Performance Analysis	Limited	Timing, area, and power analyzed
Application Scope	Basic Communication	Monitoring, embedded and IoT systems

Table 5.1 Comparison with Existing Solutions

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Summary of Achievements

In this project, we successfully designed and implemented an SPI-based data processing and alert system by following a complete VLSI design flow. The system was first developed at the RTL level using Verilog, where different modules such as the FSM, shift register, clock divider, and bit counter were created and connected together. After designing the system, it was verified using simulation in Vivado, where the correct working of signals like MOSI, MISO, clock, and control signals was confirmed through waveform analysis. This step ensured that the system was logically correct and working as expected.

After verification, the design was synthesized using Cadence Genus, where the RTL code was converted into a gate-level netlist and optimized for area, timing, and power. The synthesis results showed that the design uses a reasonable number of standard cells and meets all timing requirements with positive slack. The design was then taken to Cadence Innovus for physical implementation, where floorplanning, placement, clock tree synthesis, and routing were performed. The final layout was successfully generated without congestion or routing issues, which shows that the design is physically valid and implementable.

Overall, the system is able to perform SPI communication correctly, transfer data accurately, and generate an alert signal based on a threshold condition. The results show that the design is efficient, reliable, and well-optimized. This project demonstrates not only the working of SPI communication but also the complete journey from RTL design to physical layout, which is an important achievement in VLSI system design.

6.2 Impact on Society, Environment, and Ethical Considerations

This system can be used in real-world applications such as embedded systems, communication devices, and monitoring systems. It helps in reliable data transfer and simple decision-making, which are important in many electronic systems.

From an environmental point of view, the design consumes low power, which helps in saving energy. Low-power designs are important because electronic devices are used everywhere, and reducing power

usage helps in protecting the environment.

From an ethical point of view, this project does not involve any personal or sensitive data, so there are no privacy or misuse concerns. It is a safe system and follows proper engineering practices. The use of industry tools also makes it useful for real-world applications.

6.3 Limitations

Although the system works correctly, it has some limitations. The design currently supports only 8-bit data transfer, which limits its ability to handle larger or more complex data. The alert mechanism is based on a simple threshold comparison, which means it cannot perform advanced data analysis or intelligent decision-making. In addition, the system has been tested only through simulation and physical design tools, and it has not yet been implemented on real hardware such as an FPGA or fabricated chip.

Another limitation is that the design supports a basic SPI configuration and does not include advanced features such as multiple slave devices or different SPI modes. While the physical design has been completed using a generic technology library (GPDK 180), it does not fully represent real manufacturing conditions.

6.4 Future Work

In the future, this project can be improved by moving from a generic design environment to a real-world chip design process. Right now, the design is implemented using GPDK 180, which is mainly used for learning purposes. In the next stage, actual foundry technology files will be used, which provide real manufacturing rules and parameters.

After adapting the design to these files, further testing and optimization can be performed. Finally, the design can be sent for chip fabrication, where a real chip will be manufactured and tested. This will make the project more practical and closer to industry-level implementation.

REFERENCES

Research papers

- [1] M. Raj, “100 MHz high speed SPI master: Design, implementation and study on limitations of using SPI at high speed,” International Journal on Recent and Innovation Trends in Computing and Communication (IJRITCC), vol. 5, no. 7, pp. 697–700, 2017. [Online]. Available: https://www.academia.edu/36925980/100_MHz_High_Speed_SPI_Master_Design_Implementation_and_Study_on_Limitations_of_using_SPI_at_High_Speed
- [2] R. Jandyam, S. R. Kandi, and U. F. Mohammad, “Design and implementation of SPI module in Verilog HDL using FPGA design flow,” International Journal of Engineering Development and Research (IJEDR), vol. 5, no. 4, pp. 1090–1094, Nov. 2017. [Online]. Available: <https://rjwave.org/ijedr/papers/IJEDR1704175.pdf>
- [3] A. Srinivas, M. K. Kumar, and J. K. Bhandari, “Design and verification of serial peripheral interface,” International Journal of Engineering Development and Research (IJEDR), vol. 1, no. 3, pp. 130–136, Dec. 2014. [Online]. Available: <https://rjwave.org/ijedr/papers/IJEDR1303026.pdf>
- [4] A. K. Kummari, M. S. Reddy, and V. S. Kancharla, “Design and implementation of an SPI master controller using Verilog HDL,” in Proc. Int. Conf. Emerging Smart Computing and Informatics (ESCI), IEEE, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/10940931>
- [5] P. K. Singh, A. Kumar, and S. Verma, “Design and optimization in SPI master at the RTL level,” in Proc. Int. Conf. Intelligent Systems Design and Applications (ISDA), Springer, 2024. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-75170-7_31
- [6] “A review on ASIC synthesis flow employing two industry standard tools,” International Journal of Engineering Research & Technology (IJERT). [Online]. Available: <https://www.ijert.org/a-review-on-asic-synthesis-flow-employing-two-industry-standard-tools>
- [7] “ASIC design flow and methodology – An overview,” ResearchGate. [Online]. Available: https://www.researchgate.net/publication/336015572_ASIC_Design_Flow_And_Methodology_-_An_Overview
- [8] “Design and verification trends in VLSI systems,” STM Journals, 2025. [Online]. Available: <https://journals.stmjournals.com/jovdtt/article=2025/view=234162/>
- [9] “Advanced methodologies in VLSI design,” IEEE Xplore. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=11165748>
- [10] “Advanced RTL optimization and logic synthesis strategies for high-performance VLSI systems,” ResearchGate. [Online]. Available: https://www.researchgate.net/publication/401017807_Advanced_RTL_Optimization_and_Logic_Synthesis_Strategies_for_High-Performance_VLSI_Systems

APPENDIX

CODE SNIPPETS

FSM Code Snippet

```
module spi_fsm(  
    input clk,  
    input rst,  
    input start,  
    input done,  
    output reg load,  
    output reg shift,  
    output reg enable_counter,  
    output reg cs  
);  
  
reg [1:0] state;  
  
parameter IDLE      = 2'b00,  
           LOAD       = 2'b01,  
           TRANSFER  = 2'b10,  
           FINISH     = 2'b11;  
  
always @(posedge clk or posedge rst)  
begin  
    if(rst)  
        state <= IDLE;  
    else  
        begin  
            case(state)  
                IDLE:    if(start) state <= LOAD;  
                LOAD:    state <= TRANSFER;  
                TRANSFER: if(done) state <= FINISH;  
                FINISH:  state <= IDLE;  
                default: state <= IDLE;  
            endcase  
        end  
    end  
end  
  
always @(*)  
begin  
    load = 0;  
    shift = 0;  
    enable_counter = 0;  
    cs = 1;  
  
    case(state)
```

Bit Counter Code Snippet

```
module bit_counter(  
    input clk,  
    input rst,  
    input enable,  
    output reg [2:0] count,  
    output reg done  
);  
  
always @(posedge clk or posedge rst)  
begin  
    if(rst)  
    begin  
        count <= 0;  
        done <= 0;  
    end  
    else if(enable)  
    begin  
        if(count == 3'd7)  
        begin  
            count <= 0;  
            done <= 1;  
        end  
        else  
        begin  
            count <= count + 1;  
            done <= 0;  
        end  
    end  
    else  
        done <= 0;  
end  
  
endmodule
```

Shift Register Code Snippet

```
module spi_shift_register(  
    input clk,  
    input rst,  
    input load,  
    input shift,  
    input [7:0] data_in,  
    input miso,  
    output reg mosi,  
    output reg [7:0] shift_reg  
);  
  
// ===== SHIFT REGISTER =====  
always @(posedge clk or posedge rst)  
begin  
    if (rst)  
        shift_reg <= 8'b0;  
  
    else if (load)  
        shift_reg <= data_in;  
  
    else if (shift)  
        shift_reg <= {shift_reg[6:0], miso};  
end  
  
// ===== MOSI =====  
always @(posedge clk or posedge rst)  
begin  
    if (rst)  
        mosi <= 0;  
  
    else if (load)  
        mosi <= data_in[7]; // ? FIRST BIT READY BEFORE CLOCK  
  
    else if (shift)  
        mosi <= shift_reg[6]; // ? NEXT BIT  
end  
  
endmodule
```

Clock Divider Code Snippet

```
module clock_divider(  
    input clk,  
    input rst,  
    output reg sclk,  
    output reg tick    // ? important  
);  
  
reg [3:0] count;  
  
parameter DIV = 5;  
  
always @(posedge clk or posedge rst)  
begin  
    if (rst)  
    begin  
        count <= 0;  
        sclk <= 0;  
        tick <= 0;  
    end  
    else  
    begin  
        if (count == DIV)  
        begin  
            count <= 0;  
            sclk <= ~sclk;  
            tick <= 1;    // ? pulse  
        end  
        else  
        begin  
            count <= count + 1;  
            tick <= 0;  
        end  
    end  
end  
  
endmodule
```